

Phase 1 Database Design Report:

tket Sports Ticketing Platform

1. Introduction

This report outlines the conceptual and logical database design for the "tket" sports event reservation platform. The database has been designed using PostgreSQL, chosen for its strict compliance with SQL standards, robust data integrity constraints, and seamless integration with the planned Phase 3 Python/Django backend. The schema has been rigorously normalized to the Third Normal Form (3NF) to prevent data anomalies while maintaining high read/write performance.

2. Core Architectural Decisions & Normalization

To ensure the system remains highly performant under heavy read operations (users searching for tickets) and concurrent write operations (users booking tickets simultaneously), several strategic architectural decisions were made:

- **1-to-1 Table Splitting (The Ticket Engine):** Normalizing all sports data into a single table would cause severe data bloat. To resolve this, the core `tickets` table was strictly limited to easily searchable, high-priority attributes (e.g., date, city, price, capacity). Heavy text data and secondary metadata (e.g., tournament names, facilities) were migrated to a 1-to-1 weak entity companion table named `match_details`. This ensures the main search feed remains lightweight and loads instantly.
- **Text Field Constraints:** Rather than using unbounded `TEXT` data types, fields such as `facilities` and `description` (in reports) were capped using `VARCHAR(500)`. This acts as a defensive database constraint to prevent malicious memory bloat while providing ample space for valid input.
- **3NF Compliance:** All non-key columns strictly depend on the primary key. For example, user contact information is isolated entirely within the `users` table and referenced via foreign keys, eliminating transitive dependencies.

3. Entity Descriptions and Design Justifications

3.1. Users Table

Handles all authentication and profile data.

- **Flexible Authentication:** A table-level `CHECK` constraint (`email IS NOT NULL OR phone_number IS NOT NULL`) was implemented to support OTP/passwordless flows, allowing users to register with either method without violating database integrity.
- **Roles:** Managed via a `VARCHAR` status rather than a boolean to accommodate multiple access levels (Spectator, Support, Admin).

3.2. Tickets & Match Details Tables

Forms the catalog of the platform.

- **Capacity Integrity:** Implemented a `CHECK` constraint ensuring `remaining_capacity ≤ total_capacity`.
- **Pragmatic Denormalization:** The `facilities` attribute in `match_details` is stored as a comma-separated string rather than creating a separate many-to-many junction table. This intentional denormalization saves disk space and eliminates unnecessary `JOIN` overhead for purely descriptive text.

3.3. Reservations Table

The transactional bridge of the platform, built specifically to support the Phase 3 Redis 10-minute lock architecture.

- **Quantity Multiplier vs. Multi-Row Insertion:** Instead of generating four separate database rows when a user buys four tickets, a single row is created with a `quantity = 4` multiplier. This ensures the payment gateway only has to process one transaction per order and prevents "partial failures" if the database drops a connection mid-insert.
- **Dynamic Seat Assignment (`seat_info`):** Rather than modeling thousands of individual physical stadium chairs in the database—which creates massive overhead—seat assignments are handled as backend-generated

strings (e.g., "Row 12, Seats 1-4") allocated dynamically at the time of reservation.

3.4. Payments Table

The financial ledger.

- **Nullable Tracking Codes:** The `tracking_code` attribute intentionally lacks a `NOT NULL` constraint. This reflects real-world payment gateway flows where a transaction is initially logged as 'Pending' and the tracking code is only populated via an `UPDATE` command after a successful bank callback.

3.5. Reports Table

Manages support tickets and refund requests.

- **Optional Relationships:** The table features a mandatory foreign key to `users`, but an *optional* (nullable) foreign key to `reservations`. This flexibility allows the system to handle both specific transactional disputes (refunds) and general platform bugs (UI crashes) within a single unified table.

4. Strategic Indexing Strategy

To optimize database performance without suffering from the write-latency associated with "over-indexing," a surgical, 6-index strategy was implemented based heavily on the anticipated Phase 2 query requirements.

1. **Composite Search Index (`idx_ticket_search`):** Applied to `(sport_type, venue_city, ticket_date_time)`. Utilizing the B-Tree left-to-right rule, this allows PostgreSQL to instantly filter the primary homepage feed in a single read operation.
2. **Standard Date Index (`idx_ticket_date_time`):** A standard index was chosen over a `DATE()` expression index to natively support range queries (`BETWEEN`, `≥`, `≤`) and provide pre-sorted data for chronological UI displays.
3. **Phase 2 Target Index (`idx_ticket_teams`):** Applied to home and away teams specifically to support upcoming required data-extraction queries.

4. **Foreign Key Optimization:** Indexes were manually applied to heavily joined foreign keys (`user_id`, `ticket_id`, `reservation_id`). This eliminates full table scans during reverse lookups (e.g., an Admin querying "Which users hold tickets for Match X?").
5. **Partial Indexing for Status (`idx_res_pending`):** Instead of indexing the entire `reservation_status` column—which suffers from low cardinality—a Partial Index (`WHERE reservation_status = 'Pending'`) was created. This uses a fraction of the disk space while allowing the backend to instantly locate and expire un-paid 10-minute Redis locks.